Template

by MonotonicQueue

Order: 字符串 $\rightarrow$ 数学 $\rightarrow$ 数据结构 $\rightarrow$ 图论

仅收录较(?)常用模板，忽略大部分(?)简单模板

[可爱的封面.png] (这里本来应该有一张可爱的封面但是我不知道要放什么)

KMP

```
for(int i=2,j=0;i<=n;i++){
    while(j && s[i] != s[j+1]) j = kmp[j];
    if(s[i] == s[j+1]) j++;
    kmp[i] = j;
}
```

Manacher

```
const int N = 2e5+10;
int n,P[N],ans;
char a[N],s[N];
inline void change(){
    int k = 0;
    s[0] = '$'; s[++k] = '#';
    for(int i=1;i<=n;i++){
        s[++k] = a[i];
        s[++k] = '#';
    }
    s[++k]= '&';
    n = k;
}
void manacher(){
    int R = 0,C = 0;
    for(int i=1;i<n;i++){
        if(i < R) P[i] = min(P[C*2-i],P[C]+C-i);
        else P[i] = 1;
        while(s[i-P[i]] == s[i+P[i]]) P[i]++;
        if(i+P[i] > R){
            R = i+P[i];
            C = i;
        }
    }
}
int main(){
    scanf("%s",a+1); n = strlen(a+1);
    change(); manacher();
    for(int i=1;i<=n;i++) cout << s[i] << " ";
    cout << "\n";
    for(int i=1;i<=n;i++) cout << P[i] << " ";
    cout << "\n";
    for(int i=1;i<=n;i++) ans = max(ans,P[i]);
    printf("%d",ans-1);
    return 0;
}
```

exKMP / Z函数

```
const int N = 2e7+10;
int n,m;
char a[N],b[N];
int Z[N],p[N];
void getZ(){
```

```
    z[1] = m;
    for(int i=2,l=0,r=0;i<=m;i++){
        if(i <= r) z[i] = min(z[i-l+1],r-i+1);
        while(i+z[i] <= m && b[i+z[i]] == b[1+z[i]]) z[i]++;
        if(i+z[i]-1 > r) r = i+z[i]-1,l = i;
    }
}
void exkmp(){
    for(int i=1,l=0,r=0;i<=n;i++){
        if(i <= r) p[i] = min(z[i-l+1],r-i+1);
        while(i+p[i] <= n && a[i+p[i]] == b[1+p[i]]) p[i]++;
        if(i+p[i]-1 > r) r = i+p[i]-1,l = i;
    }
}
int main(){
    ios::sync_with_stdio(false);
    cin >> a+1 >> b+1;
    n = strlen(a+1); m = strlen(b+1);
    getz(); exkmp();
    ll ans = 0;
    for(int i=1;i<=m;i++) ans ^= 1ll*i*(z[i]+1);
    cout << ans << "\n";
    ans = 0;
    for(int i=1;i<=n;i++) ans ^= 1ll*i*(p[i]+1);
    cout << ans;
    return 0;
}
```

AC自动机

```
const int N = 1e6+10000;
int n,len,tot;
struct Node{
    int son[26];
    int end,fail;
}t[N];
void insert(char ch[]){
    int p = 0,lench = strlen(ch+1);
    for(int i=1;i<=lench;i++){
        int c = ch[i]-'a';
        if(!t[p].son[c]) t[p].son[c] = ++tot;
        p = t[p].son[c];
    }
    t[p].end++;
}
void getFail(){
    queue<int>q;
    for(int i=0;i<26;i++)
        if(t[0].son[i]) q.push(t[0].son[i]);
    while(!q.empty()){
        int x = q.front(); q.pop();
        for(int i=0;i<26;i++)
            if(t[x].son[i]){
                t[t[x].son[i]].fail = t[t[x].fail].son[i];
```

```
                q.push(t[x].son[i]);
            }else t[x].son[i] = t[t[x].fail].son[i];
    }
}
char ch[N];
int main(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        scanf("%s",ch+1);
        int lench = strlen(ch+1);
        insert(ch);
    }
    getFail();

    return 0;
}
```

后缀数组(SA)

```
struct SA{
    int n;
    string ch;
    int sa[N],rk[N],height[N],x[N],y[N],cnt[N];
    void getsa(){
        int m = 127;
        for(int i=1;i<=n;i++) cnt[x[i] = ch[i]]++;
        for(int i=1;i<=m;i++) cnt[i] += cnt[i-1];
        for(int i=n;i>=1;i--) sa[cnt[x[i]]--] = i;
        for(int k=1;k<=n;k*=2){
            int p = 0;
            for(int i=n-k+1;i<=n;i++) y[++p] = i;
            for(int i=1;i<=n;i++) if(sa[i] > k) y[++p] = sa[i]-k;

            for(int i=1;i<=m;i++) cnt[i] = 0;
            for(int i=1;i<=n;i++) cnt[x[i]]++;
            for(int i=2;i<=m;i++) cnt[i] += cnt[i-1];
            for(int i=n;i>=1;i--) sa[cnt[x[y[i]]]--] = y[i];
            swap(x,y); p = 1;
            x[sa[1]] = 1;
            for(int i=2;i<=n;i++)
                x[sa[i]] = y[sa[i]] == y[sa[i-1]] && y[sa[i]+k] == y[sa[i-1]+k]
? p : ++p;
            m = p;
            if(m >= n) break;
        }
    }
    void getheight(){
        for(int i=1;i<=n;i++) rk[sa[i]] = i;
        int k = 0;
        for(int i=1;i<=n;i++){
            if(rk[i] == 1) continue;
            if(k) k--;
            int j = sa[rk[i]-1];
            while(ch[i+k] == ch[j+k]) k++;
            height[rk[i]] = k;
        }
```

```
    }
    void init(string s){
        n = s.size();
        ch = " "+s;
        getsa(); getheight();
    }
};
```

后缀自动机(SAM)

```
const int N = 2e5+10;
int n; char ch[N];
int fa[N],son[N][26],len[N],tot=1,lst=1;
inline void insert(int c){
    int p = lst,cur = ++tot;
    len[cur] = len[p]+1;
    while(p && !son[p][c]) son[p][c] = cur,p = fa[p];
    if(!p) fa[cur] = 1;
    else{
        int q = son[p][c];
        if(len[p]+1 == len[q]) fa[cur] = q;
        else{
            int nq = ++tot;
            len[nq] = len[p]+1;
            memcpy(son[nq],son[q],sizeof(son[nq]));
            fa[nq] = fa[q];
            fa[q] = fa[cur] = nq;
            while(p && son[p][c] == q)
                son[p][c] = nq,p = fa[p];
        }
    }
    lst = cur;
}
int main(){
    scanf("%s",ch+1);
    n = strlen(ch+1);
    for(int i=1;i<=n;i++)
        insert(ch[i]-'a');
    return 0;
}
```

回文自动机(PAM)

```
const int N = 1e6+10;
int n,lst,tot=1;
struct Node{
    int len,fail,cnt;
    int son[26];
}t[N];
char s[N];
int getfail(int x,int i){
    while(s[i-t[x].len-1] != s[i]) x = t[x].fail;
    return x;
}
```

```
void insert(int x){
    int pos = getfail(lst,x),c = s[x]-'a';
    if(!t[pos].son[c]){
        t[++tot].fail = t[getfail(t[pos].fail,x)].son[c];
        t[pos].son[c] = tot;
        t[tot].len = t[pos].len+2;
    }
    /*.....*/
}
int main(){
    scanf("%s",s+1);
    n = strlen(s+1);
    t[0].fail = 1; t[1].len = -1;
    for(int i=1;i<=n;i++) insert(i);
    return 0;
}
```

```
ll fac[N],inv[N];
ll fpow(ll x,ll k){
    ll res = 1;
    while(k){
        if(k & 1)
            res = res*x%mod;
        x = x*x%mod;
        k >>= 1;
    }
    return res;
}
void init(int n = 1e6){
    fac[0] = inv[0] = 1;
    for(int i=1;i<=n;i++) fac[i] = fac[i-1]*i%mod;
    inv[n] = fpow(fac[n],mod-2);
    for(int i=n-1;i;i--) inv[i] = inv[i+1]*(i+1)%mod;
}
ll C(int x,int y){
    return fac[x]*inv[y]%mod*inv[x-y]%mod;
}
```

快速幂 & 阶乘逆元

```
ll fac[N],inv[N];
ll fpow(ll x,ll k){
    ll res = 1;
    while(k){
        if(k & 1)
            res = res*x%mod;
        x = x*x%mod;
        k >>= 1;
    }
    return res;
}
void init(int n = 1e6){
    fac[0] = inv[0] = 1;
    for(int i=1;i<=n;i++) fac[i] = fac[i-1]*i%mod;
```

```
    inv[n] = fpow(fac[n],mod-2);
    for(int i=n-1;i;i--) inv[i] = inv[i+1]*(i+1)%mod;
}
ll C(int x,int y){
    return fac[x]*inv[y]%mod*inv[x-y]%mod;
}
```

莫比乌斯反演

例题：

给定 N, M，求 $1 \leq x \leq N$，$1 \leq y \leq M$ 且 $\gcd(x, y)$ 为质数的 (x, y) 有多少对。

```
#include<bits/stdc++.h>
#define ll long long
using namespace std;
ll read(){
    ll X = 0,r = 1;
    char ch = getchar();
    while(!isdigit(ch) && ch != '-') ch = getchar();
    if(ch == '-'){ch = getchar(); r = -1;}
    while(isdigit(ch)){
        X = X*10+ch-'0';
        ch = getchar();
    }
    return X*r;
}
const int N = 1e7+10;
int pr[N/10],mu[N],sum[N],cnt;
bool vis[N];
inline void init(int n){
    mu[1] = 1;
    for(int i=2;i<=n;i++){
        if(!vis[i]){
            pr[++cnt] = i;
            mu[i] = -1;
        }
        for(int j=1;j<=cnt&&pr[j]*i<=n;j++){
            vis[i*pr[j]] = 1;
            if(i % pr[j] == 0) break;
            mu[i*pr[j]] = -mu[i];
        }
    }
    for(int i=1;i<=cnt;i++)
        for(int j=1;j*pr[i]<=n;j++) sum[j*pr[i]] += mu[j];
    for(int i=1;i<=n;i++) sum[i] += sum[i-1];
}
int main(){
    init(1e7);
    int T = read();
    while(T--){
        ll n = read(),m = read();
        if(n > m) swap(n,m);
        ll ans = 0;
        for(ll l=1,r;l<=n;l=r+1){
            r = min(n/(n/l),m/(m/l));
            ans += (n/l)*(m/l)*(sum[r]-sum[l-1]);
```

```
        }
        cout << ans << "\n";
    }
    return 0;
}
```

快速傅里叶变换FFT / 快速数论变换NTT / 多项式全家桶

```
namespace polynomial{
    const int gg = 3,ig = (mod+1)/gg;
    typedef vector<ll> poly;
    int rev[N];
    inline void NTT(poly &F,int type){
        int n = F.size();
        for(int i=0;i<n;i++)
            if(i < rev[i]) swap(F[i],F[rev[i]]);
        for(int mid=1;mid<n;mid<<=1){
            ll gn = fpow(type == 1 ? gg : ig,(mod-1)/(mid << 1));
            for(int j=0,len=mid<<1;j<n;j+=len){
                ll g0 = 1;
                for(int i=0;i<mid;i++){
                    ll x = F[i+j],y = F[i+j+mid]*g0%mod;
                    F[i+j] = (x+y)%mod;
                    F[i+j+mid] = (x-y+mod)%mod;
                    g0 = g0*gn%mod;
                }
            }
        }
        if(type != 1){
            int inv = fpow(n,mod-2);
            for(int i=0;i<n;i++) F[i] = F[i]*inv%mod;
        }
    }
    poly operator + (const poly &x,const poly &y){
        int n = x.size(),m = y.size();
        poly res(max(n,m));
        for(int i=0;i<n;i++) res[i] = x[i];
        for(int i=0;i<m;i++) res[i] = (res[i]+y[i])%mod;
        return res;
    }
    poly operator - (const poly &x,const poly &y){
        int n = x.size(),m = y.size();
        poly res(max(n,m));
        for(int i=0;i<n;i++) res[i] = x[i];
        for(int i=0;i<m;i++) res[i] = (res[i]-y[i]+mod)%mod;
        return res;
    }
    poly operator * (const poly &x,const poly &y){
        int n = x.size(),m = y.size();
        int l = 0;
        while((1 << l) < n+m-1) l++;
        for(int i=1;i<(1<<l);i++) rev[i] = (rev[i>>1]>>1) | ((i & 1) << (l-1));
        poly f(1<<l),g(1<<l);
        for(int i=0;i<n;i++) f[i] = x[i];
        for(int i=0;i<m;i++) g[i] = y[i];
```

```
        NTT(f,1); NTT(g,1);
        for(int i=0;i<(1<<l);i++) f[i] = f[i]*g[i]%mod;
        NTT(f,-1);
        f.resize(n+m-1);
        return f;
    }
    poly operator * (const ll &x,const poly &y){
        int n = y.size();
        poly res(n);
        for(int i=0;i<n;i++) res[i] = y[i]*x%mod;
        return res;
    }
    poly operator * (const poly &y,const ll &x){
        int n = y.size();
        poly res(n);
        for(int i=0;i<n;i++) res[i] = y[i]*x%mod;
        return res;
    }
    poly operator + (const ll &x,const poly &y){
        poly res = y; res[0] = (res[0]+x)%mod;
        return res;
    }
    poly operator + (const poly &y,const ll &x){
        poly res = y; res[0] = (res[0]+x)%mod;
        return res;
    }
    poly operator - (const ll &x,const poly &y){
        int n = y.size();
        poly res(n); res[0] = x;
        for(int i=0;i<n;i++) res[i] = (res[i]+mod-y[i]);
        return res;
    }
    poly operator - (const poly &y,const ll &x){
        poly res = y; res[0] = (res[0]-x+mod)%mod;
        return res;
    }
    poly polyinv(const poly &f,int len){
        if(len == 1) return {fpow(f[0],mod-2)};
        poly&& g = polyinv(f,(len+1) >> 1);
        poly tmp(len);
        for(int i=0;i<len;i++) tmp[i] = f[i];
        g = 2*g-tmp*g*g;
        g.resize(len);
        return g;
    }
    inline poly polyinv(const poly &f){
        return polyinv(f,f.size());
    }
    poly polysqrt(const poly &f,int len){
        if(len == 1){
            assert(f[0] == 1);
            //二次剩余
            return {1};
        }
        poly&& g = polysqrt(f,(len+1) >> 1);
        poly tmp(len); ll inv2 = (mod+1)/2;
        for(int i=0;i<len;i++) tmp[i] = f[i];
        g.resize(len);
```

```
        g = (tmp*polyinv(g)+g)*inv2;
        g.resize(len);
        return g;
    }
    inline poly polysqrt(const poly &f){
        return polysqrt(f,f.size());
    }
    inline poly polydev(const poly &f){
        int n = f.size(); poly res(n);
        for(int i=1;i<n;i++) res[i-1] = f[i]*i%mod;
        return res;
    }
    inline poly polyint(const poly &f){
        int n = f.size(); poly res(n);
        for(int i=1;i<n;i++) res[i] = f[i-1]*fpow(i,mod-2)%mod;
        return res;
    }
    poly polyln(const poly &f){
        poly g = polydev(f)*polyinv(f);
        g.resize(f.size());
        return polyint(g);
    }
    poly polyexp(const poly &f,int len){
        if(len == 1) return {1};
        poly&& g = polyexp(f,(len+1) >> 1);
        poly tmp(len);
        for(int i=0;i<len;i++) tmp[i] = f[i];
        g.resize(len);
        g = g*(1-polyln(g)+tmp);
        g.resize(len);
        return g;
    }
    inline poly polyexp(const poly &f){
        return polyexp(f,f.size());
    }
    inline poly polypow(const poly &f,const ll &k){
        return polyexp(polyln(f)*k);
    }
}
using namespace polynomial;
```

快速沃尔什变换 / 二进制或 与 异或卷积 / FWT

```
const int N = (1<<17)+5;
const int mod = 998244353;
int n;
ll a[N],b[N],A[N],B[N];
inline void FWTOR(ll *f,ll op){
    for(int mid=1;mid<n;mid<<=1)
        for(int len=mid<<1,j=0;j<n;j+=len)
            for(int i=0;i<mid;i++)
                f[i+j+mid] = (f[i+j]*op+f[i+j+mid])%mod;
}
inline void FWTAND(ll *f,ll op){
    for(int mid=1;mid<n;mid<<=1)
        for(int len=mid<<1,j=0;j<n;j+=len)
            for(int i=0;i<mid;i++)
```

```
                f[i+j] = (f[i+j]+f[i+j+mid]*op)%mod;
}
inline void FWTXOR(ll *f,ll op){
    for(int mid=1;mid<n;mid<<=1)
        for(int len=mid<<1,j=0;j<n;j+=len)
            for(int i=0;i<mid;i++){
                ll A0 = f[i+j],A1 = f[i+j+mid];
                f[i+j] = (A0+A1)*op%mod;
                f[i+j+mid] = (A0-A1+mod)*op%mod;
            }
}
ll fpow(ll x,ll k){
    ll res = 1;
    while(k){
        if(k & 1)
            res = res*x%mod;
        x = x*x%mod;
        k >>= 1;
    }
    return res;
}

void init(){
    for(int i=0;i<n;i++) A[i] = a[i],B[i] = b[i];
}
void work(){
    for(int i=0;i<n;i++) A[i] = A[i]*B[i]%mod;
}
void output(){
    for(int i=0;i<n;i++) cout << A[i] << " ";
    cout << "\n";
}
int main(){
    n = read(); n = 1<<n;
    for(int i=0;i<n;i++) a[i] = read();
    for(int i=0;i<n;i++) b[i] = read();
    init(); FWTOR(A,1); FWTOR(B,1);
    work(); FWTOR(A,mod-1); output();
    init(); FWTAND(A,1); FWTAND(B,1);
    work(); FWTAND(A,mod-1); output();
    init(); FWTXOR(A,1); FWTXOR(B,1);
    work(); FWTXOR(A,fpow(2,mod-2)); output();
    return 0;
}
```

平衡二叉搜索树——FHQ Tree

```
const int N = 1e5+10;
int n;
int cnt,root;
struct FHQ_Tree{
    int ls,rs;
    int pri,key,siz;
}t[N];
int newNode(int x){
```

```
    t[++cnt].pri = rand();
    t[cnt].key = x;
    t[cnt].siz = 1;
    return cnt;
}
inline void push_up(int rt){
    t[rt].siz = t[t[rt].ls].siz+t[t[rt].rs].siz+1;
}
void split(int rt,int x,int &L,int &R){
    if(!rt){L = R = 0; return ;}
    if(t[rt].key <= x){
        L = rt;
        split(t[rt].rs,x,t[rt].rs,R);
    }else{
        R = rt;
        split(t[rt].ls,x,L,t[rt].ls);
    }
    push_up(rt);
}
int merge(int L,int R){
    if(!L || !R) return L+R;
    if(t[L].pri < t[R].pri){
        t[L].rs = merge(t[L].rs,R);
        push_up(L);
        return L;
    }else{
        t[R].ls = merge(L,t[R].ls);
        push_up(R);
        return R;
    }
}
int kth(int rt,int x){
    if(t[t[rt].ls].siz+1 == x) return t[rt].key;
    if(t[t[rt].ls].siz >= x) return kth(t[rt].ls,x);
    return kth(t[rt].rs,x-t[t[rt].ls].siz-1);
}
```

李超线段树

```
const int N = 1e5+10;
const int M = 1.6e5+10;
const int mod = 39989;
int n,cnt;
struct line{
    double k,b;
}p[N];
double calc(int id,int x){
    return p[id].k*x+p[id].b;
}
void add(int x,int y,int xx,int yy){
    cnt++;
    if(x == xx) p[cnt].k = 0,p[cnt].b = max(y,yy);
    else p[cnt].k = (yy-y)*1.0/(xx-x),p[cnt].b = y-p[cnt].k*x;
}
int t[M];
#define ls rt<<1
```

```
#define rs rt<<1|1
const double eps = 1e-9;
int cmp(double x,double y){
    if(x-y > eps) return 1;
    if(y-x > eps) return -1;
    return 0;
}
void upd(int rt,int l,int r,int u){
    int &v = t[rt],mid = (l+r)>>1;
    int bmid = cmp(calc(u,mid),calc(v,mid));
    if(bmid == 1 || (bmid == 0 && u < v)) swap(u,v);
    int bl = cmp(calc(u,l),calc(v,l)),br = cmp(calc(u,r),calc(v,r));
    if(bl == 1 || (bl == 0 && u < v)) upd(ls,l,mid,u);
    if(br == 1 || (br == 0 && u < v)) upd(rs,mid+1,r,u);
}
void update(int ql,int qr,int id,int rt=1,int l=1,int r=mod){
    if(ql <= l && r <= qr){
        upd(rt,l,r,id);
    }else{
        int mid = (l+r)>>1;
        if(ql <= mid) update(ql,qr,id,ls,l,mid);
        if(qr >  mid) update(ql,qr,id,rs,mid+1,r);
    }
}
typedef pair<double,int> pdi;
pdi pmax(pdi x,pdi y){
    if(cmp(x.fi,y.fi) == 1 || (cmp(x.fi,y.fi) == 0 && x.se < y.se)) return x;
    return y;
}
pdi query(int pos,int rt=1,int l=1,int r=mod){
    if(pos < l || pos > r) return mk(0,0);
    int mid = (l+r)>>1;
    if(l == r) return mk(calc(t[rt],pos),t[rt]);
    return
pmax(mk(calc(t[rt],pos),t[rt]),pmax(query(pos,ls,l,mid),query(pos,rs,mid+1,r)));
}
```

Link Cut Tree(LCT)

```
const int N = 1e5+10;
int n,m;
struct Link_Cut_Tree{
    int fa,son[2];
    int sum,val,tag;
}c[N];
#define ls c[x].son[0]
#define rs c[x].son[1]
bool isRoot(int x){
    int fa = c[x].fa;
    return c[fa].son[0] != x && c[fa].son[1] != x;
}
inline void push_up(int x){
    c[x].sum = c[x].val^c[ls].sum^c[rs].sum;
}
inline void reverse(int x){
    if(!x) return;
    swap(ls,rs);
```

```
    c[x].tag ^= 1;
}
inline void push_down(int x){
    if(!c[x].tag) return;
    reverse(ls); reverse(rs);
    c[x].tag = 0;
}
inline void push(int x){
    if(!isRoot(x)) push(c[x].fa);
    push_down(x);
}
inline void rotate(int x){
    int y = c[x].fa,z = c[y].fa;
    bool k = c[y].son[1] == x;
    if(!isRoot(y)) c[z].son[c[z].son[1] == y] = x;
    c[x].fa = z;
    c[y].son[k] = c[x].son[k^1];
    if(c[x].son[k^1]) c[c[x].son[k^1]].fa = y;
    c[y].fa = x;
    c[x].son[k^1] = y;
    push_up(y);
}
void splay(int x){
    push(x);
    while(!isRoot(x)){
        int y = c[x].fa,z = c[y].fa;
        if(!isRoot(y))
            (c[z].son[0] == y)^(c[y].son[0] == x) ? rotate(x) : rotate(y);
        rotate(x);
    }
    push_up(x);
}
void access(int x){
    for(int child=0;x;child=x,x=c[x].fa){
        splay(x);
        rs = child;
        push_up(x);
    }
}
void makeroot(int x){
    access(x); splay(x); reverse(x);
}
void split(int x,int y){
    makeroot(x);
    access(y);
    splay(y);
}
void link(int x,int y){
    makeroot(x); c[x].fa = y;
}
void cut(int x,int y){
    split(x,y);
    if(c[y].son[0] != x || rs) return;
    c[y].son[0] = 0; c[x].fa = 0;
    push_up(x);
}
int findroot(int x){
    access(x); splay(x);
```

```
    while(ls) push_down(x),x = ls;
    return x;
}
int main() {
    n = read(); m = read();
    for(int i=1;i<=n;i++)
        c[i].sum = c[i].val = read();
    while(m--){
        int op = read();
        int a = read(),b = read();
        if(op == 0){
            split(a,b);
            cout << c[b].sum << "\n";
        }else if(op == 1){
            if(findroot(a) != findroot(b))
                link(a,b);
        }else if(op == 2){
            cut(a,b);
        }else{
            splay(a); c[a].val = b;
        }
    }
    return 0;
}
```

KTT

```
const int N = 4e5+10;
const ll inf = 2e18;
int n,Q,a[N];
namespace KTT{
struct Line{
    ll k,b;
    Line(ll k=0,ll b=0): k(k),b(b){}
    Line operator+(const Line &o)const{
        return Line(k+o.k,b+o.b);
    }
};
ll tag[N<<2];
pair<Line,ll> max(Line a,Line b){
    //求出两条直线 a,b 在 x = 0 处的 max 以及这个 max 会改变的最小 x
    if(a.k < b.k || (a.k == b.k && a.b < b.b)) swap(a,b);
    if(a.b >= b.b) return mk(a,inf);
    return mk(b,(b.b-a.b)/(a.k-b.k));
}
struct node{
    Line sum,lmx,rmx,mx;
    ll x;
    node operator+(const node &o)const{
        node res;
        res.x = min(x,o.x);
        auto tmp = max(lmx,o.lmx+sum);
        res.lmx = tmp.fi; res.x = min(res.x,tmp.se);
        tmp = max(o.rmx,rmx+o.sum);
        res.rmx = tmp.fi; res.x = min(res.x,tmp.se);
        res.sum = sum+o.sum;
```

```
        tmp = max(mx,o.mx); res.x = min(res.x,tmp.se);
        tmp = max(tmp.fi,rmx+o.lmx);
        res.x = min(res.x,tmp.se);
        res.mx = tmp.fi;
        return res;
    }
}t[N<<2];
#define ls rt<<1
#define rs rt<<1|1
void build(int rt,int l,int r){
    if(l == r){
        t[rt].sum = t[rt].lmx = t[rt].rmx = t[rt].mx = Line(1,a[l]);
        t[rt].x = inf;
        return ;
    }
    int mid = (l+r) >> 1;
    build(ls,l,mid);
    build(rs,mid+1,r);
    t[rt] = t[ls]+t[rs];
}
void change(int rt,ll d){
    tag[rt] += d; t[rt].x -= d;
    t[rt].lmx.b += t[rt].lmx.k*d;
    t[rt].rmx.b += t[rt].rmx.k*d;
    t[rt].mx.b += t[rt].mx.k*d;
    t[rt].sum.b += t[rt].sum.k*d;
}
void upd(int rt,int l,int r,ll d){
    if(t[rt].x < d){
        d += tag[rt];
        tag[rt] = 0;
        int mid = (l+r) >> 1;
        upd(ls,l,mid,d);
        upd(rs,mid+1,r,d);
        t[rt] = t[ls]+t[rs];
    }else change(rt,d);
}
inline void push_down(int rt){
    if(tag[rt]){
        change(ls,tag[rt]);
        change(rs,tag[rt]);
        tag[rt] = 0;
    }
}
void update(int rt,int l,int r,int ql,int qr,ll d){
    if(ql <= l && r <= qr) return upd(rt,l,r,d);
    push_down(rt);
    int mid = (l+r) >> 1;
    if(ql <= mid) update(ls,l,mid,ql,qr,d);
    if(qr >  mid) update(rs,mid+1,r,ql,qr,d);
    t[rt] = t[ls]+t[rs];
}
node query(int rt,int l,int r,int ql,int qr){
    if(ql <= l && r <= qr) return t[rt];
    push_down(rt);
    int mid = (l+r) >> 1;
    if(qr <= mid) return query(ls,l,mid,ql,qr);
```

```
    if(ql >  mid) return query(rs,mid+1,r,ql,qr);
    return query(ls,l,mid,ql,qr)+query(rs,mid+1,r,ql,qr);
}
}
int main(){
    n = read(); Q = read();
    for(int i=1;i<=n;i++) a[i] = read();
    KTT::build(1,1,n);
    while(Q--){
        int op = read(),l = read(),r = read();
        if(op == 1){
            int x = read();
            KTT::update(1,1,n,l,r,x);
        }else cout << max(KTT::query(1,1,n,l,r).mx.b,0ll) << "\n";
    }
    return 0;
}
```

单源最短路(Dijkstra算法)

```
void dijkstra(ll *dis,int s){
    priority_queue<pair<ll,int>> q;
    memset(flag,0,sizeof(flag));
    for(int i=1;i<=n;i++) dis[i] = 1e18;
    q.push(mk(0,s));
    dis[s]  = 0;
    while(!q.empty()){
        auto [_,x] = q.top(); q.pop();
        if(flag[x]) continue;
        flag[x] = true;
        for(auto [y,w] : G[x]){
            if(dis[y] > dis[x]+w){
                dis[y] = dis[x]+w;
                q.push(mk(-dis[y],y));
            }
        }
    }
}
```

O(1) 树上最近公共祖先LCA

```
const int N = 5e5+10;
int n,m,S;
vector<int> G[N];
int dfn[N],cnt;
int st[N][20];
inline int getmn(int x,int y){
    return dfn[x] < dfn[y] ? x : y;
}
void dfs(int x,int fa){
    st[dfn[x] = ++cnt][0] = fa;
    for(int y : G[x]){
        if(y == fa) continue;
        dfs(y,x);
```

```
    }
}
int Lg[N];
inline void st_init(){
    for(int j=1;j<=18;j++)
        for(int i=1;i+(1<<j)-1<=n;i++)
            st[i][j] = getmn(st[i][j-1],st[i+(1<<(j-1))][j-1]);
    for(int i=2;i<=n;i++) Lg[i] = Lg[i>>1]+1;
}
int lca(int x,int y){
    if(x == y) return x;
    x = dfn[x]; y = dfn[y];
    if(x > y) swap(x,y);
    x++; int lg = Lg[y-x+1];
    return getmn(st[x][lg],st[y-(1<<lg)+1][lg]);
}
int main(){
    n = read(); m = read(); S = read();
    for(int i=1;i<n;i++){
        int u = read(),v = read();
        G[u].push_back(v);
        G[v].push_back(u);
    }
    dfs(S,0); st_init();
    while(m--){
        int u = read(),v = read();
        cout << lca(u,v) << "\n";
    }
    return 0;
}
```

圆方树

```
const int N = 2e5+10;
int n,m,cnt;
//Initially cnt = n!!!
vector<int> G[N],E[N];
int dfn[N],low[N],num;
int stk[N],top;
void tarjan(int x){
    low[x] = dfn[x] = ++num;
    stk[++top] = x;
    for(int y : G[x]){
        if(!dfn[y]){
            tarjan(y);
            low[x] = min(low[x],low[y]);
            if(low[y] == dfn[x]){
                cnt++;
                int z = 0;
                do{
                    z = stk[top--];
                    E[cnt].push_back(z);
                    E[z].push_back(cnt);
                }while(z != y);
                E[cnt].push_back(x);
                E[x].push_back(cnt);
            }
```

```
        }else low[x] = min(low[x],dfn[y]);
    }
}
void clear(){
    for(int i=1;i<=n;i++) G[i].clear();
    for(int i=1;i<=2*n;i++) E[i].clear();
    top = num = tot = 0; cnt = n;
    for(int i=1;i<=n;i++) low[i] = dfn[i] = 0;
}
```

网络最大流MF(Dinic算法)

```
namespace MF{
    const int N = 210;
    const int M = 10100;
    const ll inf = 1e18;
    int n,m,S,T,tot=1;
    int head[N],to[M],nxt[M];
    ll edge[M];
    inline void add_edge(int u,int v,int z){
        to[++tot] = v; nxt[tot] = head[u];
        head[u] = tot; edge[tot] = z;
        to[++tot] = u; nxt[tot] = head[v];
        head[v] = tot; edge[tot] = 0;
    }
    int now[N],dep[N];
    bool bfs(){
        memset(dep,0,sizeof(dep));
        queue<int> q;
        while(!q.empty()) q.pop();
        dep[S] = 1; now[S] = head[S];
        q.push(S);
        while(!q.empty()){
            int x = q.front(); q.pop();
            for(int i=head[x];i;i=nxt[i]){
                int y = to[i];
                if(!edge[i] || dep[y]) continue;
                dep[y] = dep[x]+1;
                now[y] = head[y];
                if(y == T) return true;
                q.push(y);
            }
        }
        return false;
    }
    ll dinic(int x,ll flow){
        if(x == T) return flow;
        ll rest = flow;
        for(int i=now[x];i && rest;i=nxt[i]){
            now[x] = i;
            int y = to[i];
            if(dep[y] != dep[x]+1 || !edge[i]) continue;
            ll k = dinic(y,min(rest,edge[i]));
            if(!k) dep[y] = 0;
            edge[i] -= k;
```

```
            edge[i^1] += k;
            rest -= k;
        }
        return flow-rest;
    }
    ll MF(){
        ll ans = 0;
        while(bfs()) ans += dinic(S,inf);
        return ans;
    }
}
```

最小费用最大流MCMF(SPFA+EK)

```
namespace MCMF{
    const int N = 3e6+10;
    const int M = 3e6+10;
    const ll inf = 1e18;
    int tot=1,n,S,T;
    int head[N],to[M],nxt[M];
    ll edge[M],cost[M];
    inline void add_edge(int u,int v,int w,int c){
        to[++tot] = v; nxt[tot] = head[u]; head[u] = tot;
        edge[tot] = w; cost[tot] = c;
        to[++tot] = u; nxt[tot] = head[v]; head[v] = tot;
        edge[tot] = 0; cost[tot] = -c;
    }
    bool vis[N];
    int pre[N];
    ll d[N],incf[N];
    bool SPFA(){
        memset(vis,0,sizeof(vis));
        for(int i=1;i<=n;i++) d[i] = -inf;
        incf[S] = inf; d[S] = 0;
        queue<int> q;
        while(!q.empty()) q.pop();
        q.push(S);
        while(!q.empty()){
            int x = q.front(); q.pop(); vis[x] = false;
            for(int i=head[x];i;i=nxt[i]){
                int y = to[i];
                if(!edge[i]) continue;
                if(d[y] < d[x]+cost[i]){
                    d[y] = d[x]+cost[i];
                    incf[y] = min(incf[x],edge[i]);
                    pre[y] = i;
                    if(!vis[y]) q.push(y),vis[y] = true;
                }
            }
        }
        return d[T] != -inf;
    }
    ll maxflow,ans;
    void update(){
        int x = T;
        maxflow += incf[T];
        ans += incf[T]*d[T];
```

```
        while(x != S){
            int i = pre[x];
            edge[i] -= incf[T];
            edge[i^1] += incf[T];
            x = to[i^1];
        }
    }
    ll MCMF(){
        while(SPFA()) update();
        return ans;
    }
}
```

最小割树

```
const int N = 510;
const int M = 2e4+10;
const ll inf = 1e18;
int n,m;
vector<pii> G[N];
namespace GHT{
int S,T,tot=1;
int head[N],to[M],nxt[M];
ll edge[M];
inline void add_edge(int u,int v,int z){
    to[++tot] = v; nxt[tot] = head[u];
    head[u] = tot; edge[tot] = z;
    to[++tot] = u; nxt[tot] = head[v];
    head[v] = tot; edge[tot] = 0;
}
void rebuild(int s,int t){
    S = s,T = t;
    for(int i=2;i<=tot;i+=2){
        edge[i] += edge[i^1];
        edge[i^1] = 0;
    }
}
int now[N],dep[N];
bool bfs(){
    memset(dep,0,sizeof(dep));
    queue<int> q;
    while(!q.empty()) q.pop();
    dep[S] = 1; now[S] = head[S];
    q.push(S);
    while(!q.empty()){
        int x = q.front(); q.pop();
        for(int i=head[x];i;i=nxt[i]){
            int y = to[i];
            if(!edge[i] || dep[y]) continue;
            dep[y] = dep[x]+1;
            now[y] = head[y];
            if(y == T) return true;
            q.push(y);
        }
    }
    return false;
}
```

```
ll dinic(int x,ll flow){
    if(x == T) return flow;
    ll rest = flow;
    for(int i=now[x];i && rest;i=nxt[i]){
        now[x] = i;
        int y = to[i];
        if(dep[y] != dep[x]+1 || !edge[i]) continue;
        ll k = dinic(y,min(rest,edge[i]));
        if(!k) dep[y] = 0;
        edge[i] -= k;
        edge[i^1] += k;
        rest -= k;
    }
    return flow-rest;
}
ll MF(){
    ll ans = 0;
    while(bfs()) ans += dinic(S,inf);
    return ans;
}
bool flag[N],vis[N];
void dfs(int x){
    vis[x] = true;
    for(int i=head[x];i;i=nxt[i]){
        int y = to[i];
        if(!vis[y] && edge[i])
            dfs(y);
    }
}
void solve(vector<int> v){
    if(v.size() == 1) return ;
    int s = v[0],t = v[1];
    rebuild(s,t);
    int val = MF();
    G[s].emplace_back(t,val);
    G[t].emplace_back(s,val);
    for(int i=0;i<=n;i++) vis[i] = false;
    dfs(s);
    vector<int> vl,vr;
    for(int x : v)
        if(vis[x]) vl.push_back(x);
        else vr.push_back(x);
    solve(vl); solve(vr);
}
}
int fa[N][12],mn[N][12],dep[N];
void dfs(int x,int ft){
    dep[x] = dep[ft]+1;
    for(int i=1;i<=10;i++)
        fa[x][i] = fa[fa[x][i-1]][i-1],
        mn[x][i] = min(mn[x][i-1],mn[fa[x][i-1]][i-1]);
    for(auto [y,w] : G[x]){
        if(y == ft) continue;
        fa[y][0] = x;
        mn[y][0] = w;
        dfs(y,x);
    }
}
```

```cpp
int get(int x,int y){
    if(dep[x] < dep[y]) swap(x,y);
    int res = 1e9;
    for(int i=10;~i;i--)
        if(dep[fa[x][i]] >= dep[y])
            res = min(res,mn[x][i]),x = fa[x][i];
    if(x == y) return res;
    for(int i=10;~i;i--)
        if(fa[x][i] != fa[y][i])
            res = min(res,min(mn[x][i],mn[y][i])),
            x = fa[x][i],y = fa[y][i];
    return min(res,min(mn[x][0],mn[y][0]));
}
int main(){
    n = read(); m = read();
    for(int i=1;i<=m;i++){
        int u = read(),v = read(),w = read();
        GHT::add_edge(u,v,w);
        GHT::add_edge(v,u,w);
    }
    vector<int> v;
    for(int i=1;i<=n;i++) v.push_back(i);
    GHT::solve(v);
    dfs(1,0);
    int Q = read();
    while(Q--){
        int x = read(),y = read();
        cout << get(x,y) << "\n";
    }
    return 0;
}
```